

Ans to the Ques - 1

A protected class members in Java is accessible

1. Within the same class
2. By any subclass (child) class, even if it's in different package.
3. Within the same class by any packages.

Protected is more permissive than private but less than public.

Access rules depends on package and inheritance.

Example - 1: Same Class

Parent . Java :

```
package mypackage;

public class Parent {
    protected int number = 10;
}
```

Child . Java : Access via inheritance

```
package package mypackage;

public class extends Parent {
    public void shownumber() {
        System.out.println(number);
    }

    public static void main(String[] args) {
        Child c = new Child();
        c.show number;
    }
}
```

}

Q Another Class : Access without inheritance

```
package mypackage;
```

```
public class AnotherClass {
```

```
    public static void main(String[] args)
```

```
    {  
        Parent p = new Parent();
```

```
        System.out.println(p.number);  
    }
```

String[] args) throws IOException {

// ...
}

Example-2: Different Package

```
import mypackage.parent;
```

```
package otherpackage;
```

```
import mypackage.parent;
```

```
public
```


Example - 2: Different Package

Child. Java: Access via inheritance

package otherpackage;
import mypackage.Parent;

public class Child extends Parent {

public void showNumber () {
System.out.println("number");

}

public static void main (String[] args)

Child c = new Child();

c.showNumber();
System.out.println("Child");

~~System.out.println("Child");~~

~~System.out.println("Child");~~

~~Child;~~

..

Another class: Access of with inheritance

package other package;

import package Parent; imported

public class Another class {

... public static void main (String[] args)

Parent p = new Parent();

System.out.println (p.number) // compile time

... abouts located over font error

?

...not

...not located

...not

...not located

Same package with inheritance

Different

...not

..

...not

...not

...not

Ans to the Ques 2

Abstract Class: A class declared with abstract keyword can may have abstract method (with body) + concrete methods (with body)

Interface: A reference type in Java that have abstract method, default method, static methods, constants

Abstract Class	Interface
Multiple inheritance not allowed	Can Implement multiple Interface
Abstract + Concrete Method	Only abstract method default method with body
Instance Variable	Only constant
Constructors allowed	No constructor allowed

Use abstract class when,

- i) Need to share code (methods and fields) among related classes
- ii) Classes share common state and behaviour
- iii) Do not need multiple inheritance

Use interface when:

- i) Want to enforce a contract (methods) without caring about implementation
- ii) Need multiple inheritance
- iii) Changes maybe unrelated but share behaviour

Ans to the Ques-3

Encapsulation in java is the process of restricting direct access to class members (variable) by declaring them private and providing public setter and getter methods.

i) Ensures data security by preventing unauthorized access.

ii) Ensures data integrity by allowing validation before updating values.

public class BankAccount {

private String accountNumber;

private double balance;

public void setAccountNumber(String ac) {

if (ac == null || ac.isEmpty()) {

System.out.println("Invalid account number");

} else {

this.accountNumber = accountNumber;

}

}

public String getAccountNumber() {

return accountNumber;

}

private double balance;

} else {

System.out.println("Invalid account number");

}

}

```
public void setInitialBalance(double balance)
{
    if (balance < 0)
    {
        System.out.println("Balance can't be negative");
    }
    else
    {
        this.balance = balance;
    }
}
```

```
public double getBalance()
{
    return balance;
}
```

```
}
```

```
public void deposit(double amount)
{
    if (amount > 0)
    {
        balance += amount;
    }
    else
    {
        System.out.println("Deposit must be positive");
    }
}
```

```
}
```



```
public void withdraw(double amount){  
    if (amount > 0 & amount <= balance){  
        balance -= amount;
```

```
    } else
```

```
        System.out println("Invalid withdraw  
amount!");  
    }
```

```
}
```

```
psvm(String[] args){
```

Lab No: 4

How JDBC works:

JDBC is an API that allows java applications to communicate with relational databases. It provides a standard interface to:

1. Connect to a database
2. Send SQL commands
3. Retrieve Results
4. Handles Errors.

Key components:

① Driver Manager: Loads and manages database drivers.

② Connection: Represents a connection to Databases

③ Statement: Sends SQL queries to the database

④ ResultSet: Holds the result of a query.

Steps to Execute a Select Query and Fetch Results:

- ① Load the JDBC driver
- ② Establish a connection
- ③ Create a statement
- ④ Execute the query
- ⑤ Process the result row by row
- ⑥ Close resources
- ⑦ Handle exceptions

Code:

```
public class main {
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Connection conn=null;
```

```
        Statement stat=null;
```

```
        ResultSet rs=null;
```

```
        try {
```

```
            conn = DriverManager.get
```

```
                Connection ("jdbc:mysql://
```

```
                    localhost:3306/school", "root",  
                    "password",
```

try {

~~conn~~

conn =

stmt = conn.createStatement();

rs = stmt.executeQuery("Select id,

name, grade From student");

while(rs.next()) {

int id = rs.getInt("id");

String name = rs.getString("name");

double grade = rs.getDouble("grade");

System.out.println("ID: " + id + ", Name: " +

name + ", grade: " + grade);

} catch (SQLException e) {

System.err.println("Database

error: " + e.getMessage);

} finally {

try {

if (rs != null) rs.close();

if (stmt != null) rs.

if (conn != null) conn.close();

} catch (SQLException e) {

System.err.println("Error: "

} }

Lab No: 5

How a ~~set~~ Servlet Controller
Manage Flow?

in Java EE MVC Setup?

1 Servlet (Controller)

i) Handles incoming HTTP
requests

ii) Interacts with model
objects to process data/
business logic.

iii) Forwards the data to a
view (JSP) for rendering

2 Model

i) Java classes that hold
data or business logic

3 View (JSP)

i) Receives data from the
controller via request attributes

ii) Generates HTML responses.

Model class

```
public class User {  
    private String name;  
    private int age;  
  
    public User (String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String getName() {return name;}  
    public int getAge() {return age;}  
}
```

Servlet Controller

```
import java.io.IOException;  
import jakarta.servlet.*;  
import jakarta.servlet.http.*;  
  
public class UserServlet extends HttpServlet {  
    @Override,  
    protected void doGet(HttpServletRequest  
        request, HttpServletResponse response)  
        throws ServletException, IOException {  
        User user = new User ("Aline", 22);  
        request.setAttribute("user", user);  
        RequestDispatcher dispatcher =  
            request.getRequestDispatcher("user.js");  
    }  
}
```


dispatcher.forward(request, response);

}

}

JSP View (user.jsp)

```
<%@ page contentType="text/html;
    charset=UTF-8" %>
```

<html>

<head><title>User Info </title></head>

<body>

<h2>User details </h2>

<p>Name: \${user.name}</p>

<p>Age: \${user.age}</p>

</body>

</html>

Lab - 60

Why Prepared Statement is Better than Statement:

Statement	Prepared Statement
Parse SQL every time	SQL is precompiled can be used multiple times.
Vulnerable to SQL Injection if user input is concatenated	Safely handles input placeholders.
No parameter binding	Yes
Low Reusability	High Reusability

Code:

```
import java.sql.*;

public class Main {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/
                      school";
        String username = "root";
        String password = "password";
        Connection conn = null;
        PreparedStatement pstmt = null;

        Conn = DriverManager.getConnection(url,
                                             username, password);

        String sql = "insert into Students
                      (id, name, grade) values (?,?,?)"

        pstmt = conn.prepareStatement(sql);
        pstmt.setInt(1, 101);
        pstmt.setString(2, "Bob");
        pstmt.setInt(3, 85);
        int rows = pstmt.executeUpdate();

        pstmt.close();
        Conn.close();
    }
}
```

Lab No: 87

ResultSet:

i) A `ResultSet` is an object returned by executing a SQL `SELECT` query using `JDBC`

ii) It represents table of data from database

iii) You can `iterate` row by row

Key Methods:

i) `next`

ii) `getString()`

iii) `getInt`

Code:

```
public class main {  
    public static void main(String[] args) {  
        String url = "jdbc:mysql://localhost:3306/  
                        school";  
        String password = "123";  
  
        Conn = DriverManager.getConnection(url, name, pass);  
        stmt = conn.createStatement();  
        rs = stmt.executeQuery("select id, name, grade  
                                from student");  
        while (rs.next()) {  
            int id = rs.getInt("id");  
            String n = rs.getString("name");  
            double d = rs.getDouble("grade");  
        }  
    }  
}
```


~~Lab-9~~

Lab-No : 8

How Spring Boot Simplifies RESTful Service Development?

- ① Auto configuration
- ② Embedded Server
- ③ ~~Annotations~~ Annotations
- ④ JSON Handling
- ⑤ Dependency Management

Code:

REST Controller:

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    private List<User> users = new ArrayList<>
```

@GetMapping

```
public List<User> getAllUsers() {
    return users;
}
```

}

@PostMapping

```
public String addUser(@RequestBody User user) {
    users.add(user);
    return "User added";
}
```

}

JSON:

```
{ "name": "Alice",
  "age": 22
}
```